ASTANA IT UNIVERSITY

School of Software Engineering
Software Engineering programme · Group SE-2430

Smart Finance

Development of an Intelligent Application for Personal Finance Planning and Expense Analysis

Individual implementation report: mobile client, backend API, data storage and integration.

Submitted by	Project team
Kaber Daryn	Kaber Daryn · Nuriman Aisultan · Akhmetov Dauren
Supervisor	Bakayeva Bakyt, Senior Lecturer
Report date	24 October 2026

Executive summary

Smart Finance is a team prototype for recording personal income and expenses, reviewing spending and planning budgets and savings goals. My assigned contribution covers the client, server API, database and integration with the analytical module. This report presents the implementation and evidence available for pre-defence, rather than claiming completion of every final-diploma evaluation.

The submitted demonstration materials show a responsive mobile interface, a local Python server, successful account registration and transaction creation/retrieval, SQLite table structure and a successful 51-test run. A separate receipt-extraction pilot provides preliminary field-match and processing-time results. Planned model-quality and usability evaluations are distinguished from completed checks.

Evidence basis: supplied source archive, team role statements, receipt-evaluation JSON and screenshots submitted for this report [1–4].

1. Project Aim, Scope and Individual Contribution

1.1 Aim and engineering tasks

The project aim is to integrate transaction recording, expense summaries and planning tools with editable analytical assistance. The principal workflow is capture → validation → review → confirmation → storage. Receipt extraction proposes fields; it does not authorise an expense or replace user confirmation.

Requirement	Implementation / evidence
Account separation	Authenticated API and per-user records; registration shown with HTTP 201.
Income and expense recording	Typed transaction payload, integer minor-unit amounts and dated records; expense round-trip demonstrated.
History and summaries	Search/filter controls, transaction list and spending views shown in the interface.
Receipt-assisted entry	Consent, editable extracted fields, validation and subsequent transaction-list state shown.
Financial planning	Budget and savings-goal support in source; supplied Planning screen shows an empty-budget state.
Analytical assistance	Description-category baseline, simple forecast and confirmed merchant-category reuse in source; real-world quality evaluation pending.

1.2 Allocation of work

Kaber Daryn: system architecture, Flutter client, backend API, relational storage, integration and local deployment. Nuriman Aisultan: UX/UI, test planning and quality assurance. Akhmetov Dauren: analytical module, expense categorisation and forecasting. This allocation follows the supplied project documents; the common source snapshot does not independently establish line-by-line authorship [1, 4].

The term intelligent in the diploma title refers to assisted receipt interpretation and category suggestions. The prototype does not introduce a new learning algorithm or provide autonomous financial advice.

2. Architecture and Data Handling

The source separates the client, HTTP API, storage and analytical functions. The backend uses Python ThreadingHTTPServer and JSON endpoints. SQLite supports local execution; a PostgreSQL adapter and Docker Compose configuration are supplied for an alternative deployment [1].

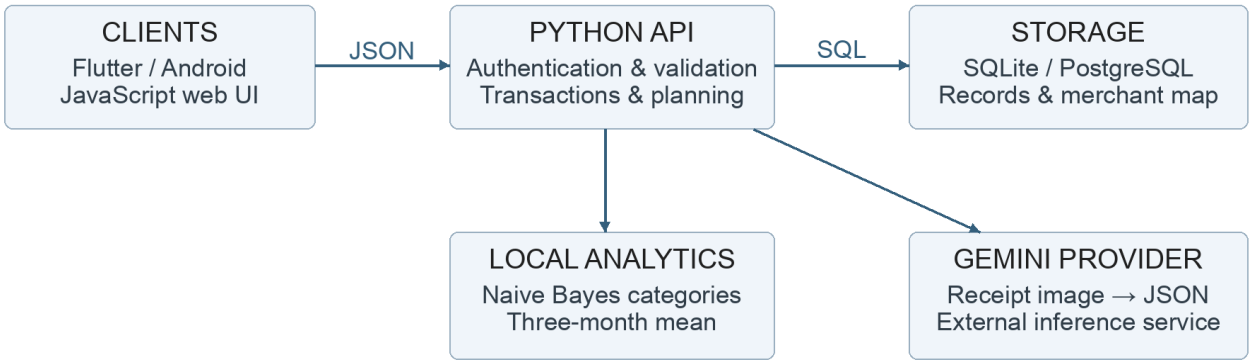


Figure 1. Component architecture of the supplied source snapshot.

2.1 Storage design

Transaction records contain id, kind, amount_minor, category, description, created_at and user_id. Amounts are stored as integer minor units; 125000 represents KZT 1250.00. Authentication and queries associate private records with the current user. Passwords are salted and hashed; stored session tokens are hashed in the source implementation.



Figure 2. SQLite transactions schema printed in the submitted PowerShell session; surrounding desktop content cropped.

The SQLite screenshot supports the demonstrated local storage structure, not an independently tested PostgreSQL deployment. The supplied mobile-layout screens are described by their visible state; their operating system and native-versus-browser execution mode are not established by the images alone.

3. Local Execution and Account Registration

The demonstration uses Windows PowerShell and a local API at http://127.0.0.1:8765. The submitted server terminal shows the port assignment and module-based launch. The health request returns status ok [2].

```
$env:PORT = "8765"
python -m backend.server
Invoke-RestMethod http://127.0.0.1:8765/health
```

```
Smart Finance: http://127.0.0.1:8765
```

Figure 3. Local server address after the module-based launch; terminal excerpt.

```
status
------
ok
```

Figure 4. Health-response excerpt: status ok.

3.1 Registration evidence

A separate PowerShell request posts a generated demonstration username and password to /auth/register. The response status is HTTP 201. The screenshot then constructs an Authorization header from the returned session token, without printing its value.

```
POST /auth/register — HTTP 201
PS C:\Users\242249\Downloads\Smart_Finance_Flutter_Design_FIXED\Smart_Finance_Flutt
Создан тестовый пользователь: evdemo_1024100041
```

Figure 5. Registration result and generated test username; excerpt cropped from the submitted terminal.

The PowerShell security warning visible in the full request screenshots concerns Invoke-WebRequest content parsing. It is not an API failure status. The displayed API results are used as the evidence; no credentials or token values are reproduced in this report.

4. API Transaction Round-Trip

The API demonstration submits a fictional expense with kind expense, amount_minor 125000, category Groceries and description “Fictional test transaction”. The server returns HTTP 201 and a JSON record with id 3. A subsequent authenticated GET /transactions returns HTTP 200 and contains the same record.

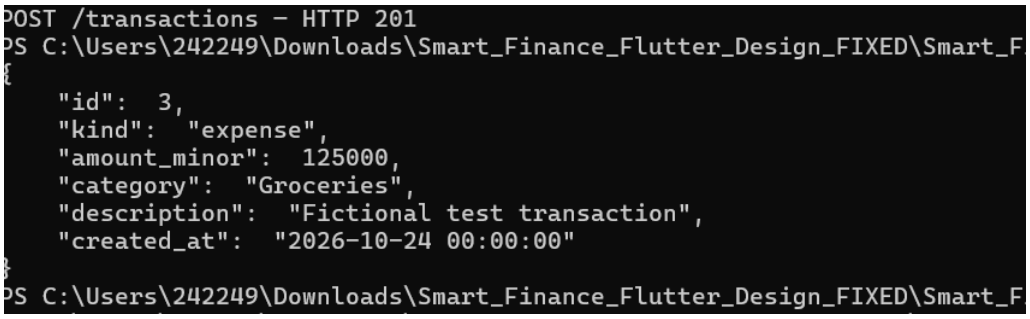


Figure 6. POST /transactions result and saved fictional record. Excerpt cropped; unrelated warning text omitted.

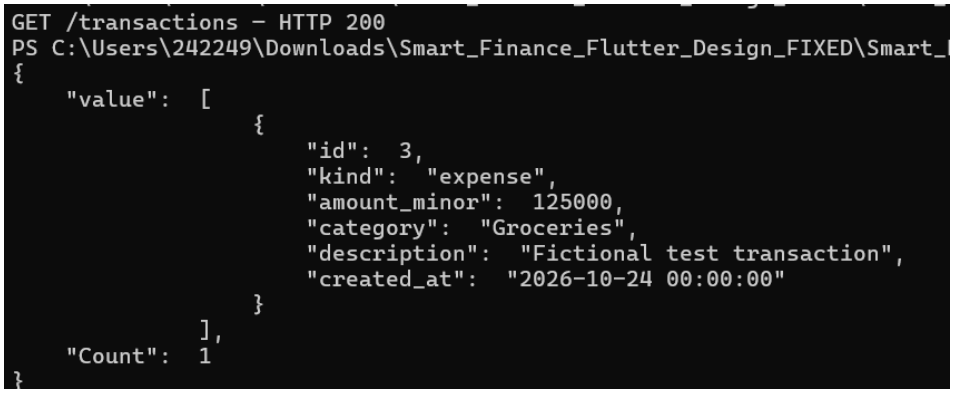


Figure 7. GET /transactions result containing the created record. Excerpt cropped from the submitted terminal.

This sequence supports account-based transaction creation and immediate retrieval. It is separate from the mobile-screen account and does not demonstrate persistence across a process restart, concurrent load or bank integration. The displayed date is a test transaction date, not the report’s receipt-ground-truth label.

5. User Interface: Entry and Overview

The interface provides sign-in and account creation, an overview balance and a route to transaction entry. The screenshots below are supplied demonstration states, not independently observed native-device performance measurements [2].

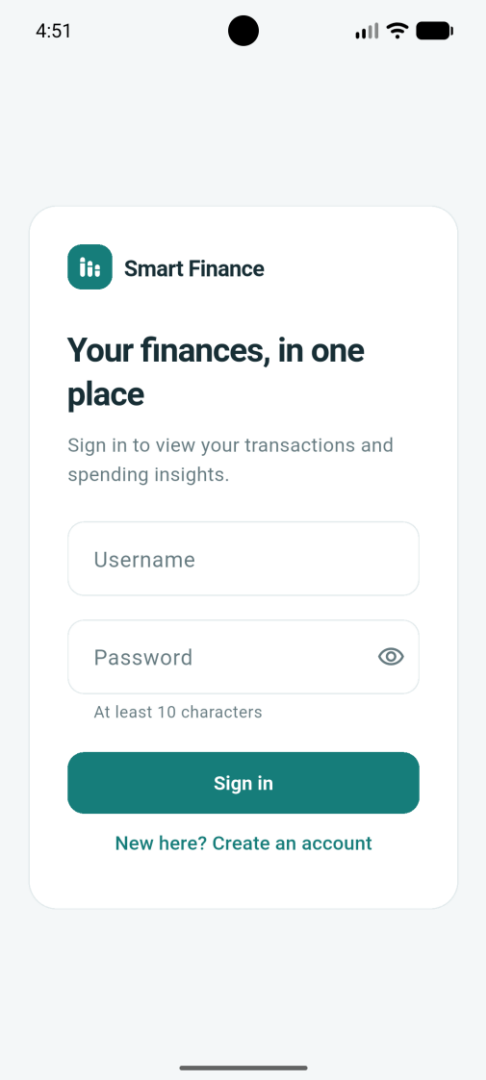


Figure 8. Sign-in screen with empty credential fields.

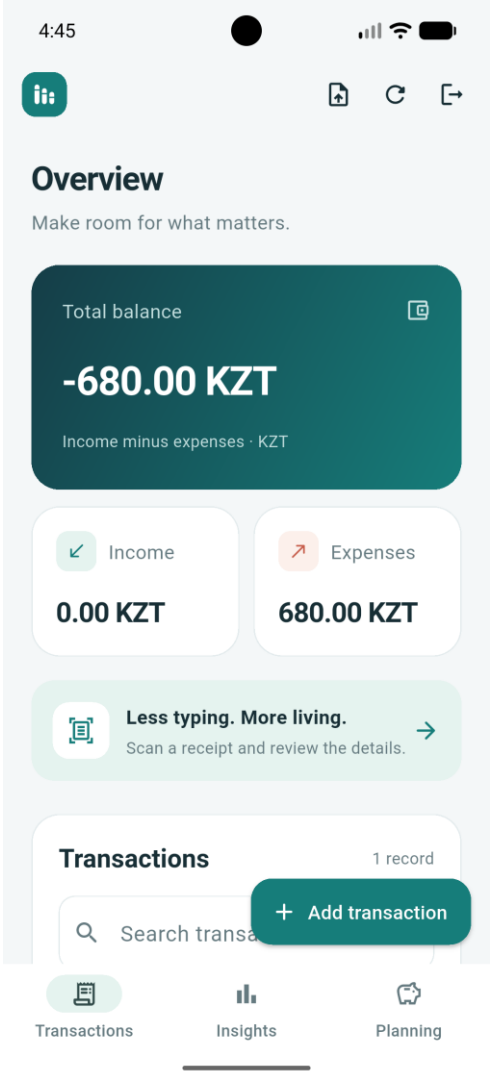


Figure 9. Overview after a KZT 680.00 expense and no income.

In the overview state, the displayed balance is negative because recorded expenses exceed recorded income. It is an application-ledger balance, not a connected bank-account balance. The interface's promotional wording about less typing is not treated as a measured usability result.

6. Receipt Capture and Review

The receipt flow asks for agreement before enabling image capture or gallery selection. The review screen displays an extracted merchant and amount and allows user correction. A missing date remains available for manual entry [2].

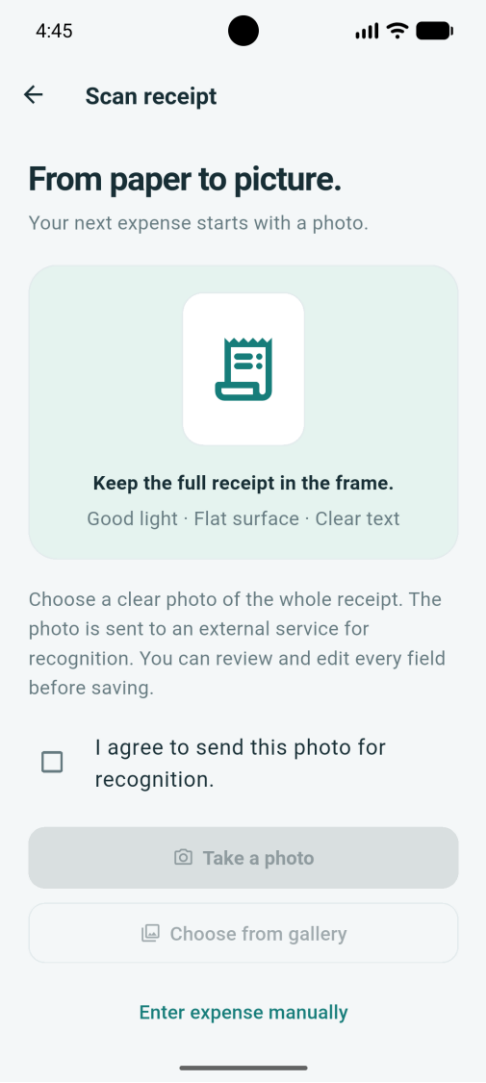


Figure 10. External-service notice with consent unchecked; capture actions disabled.

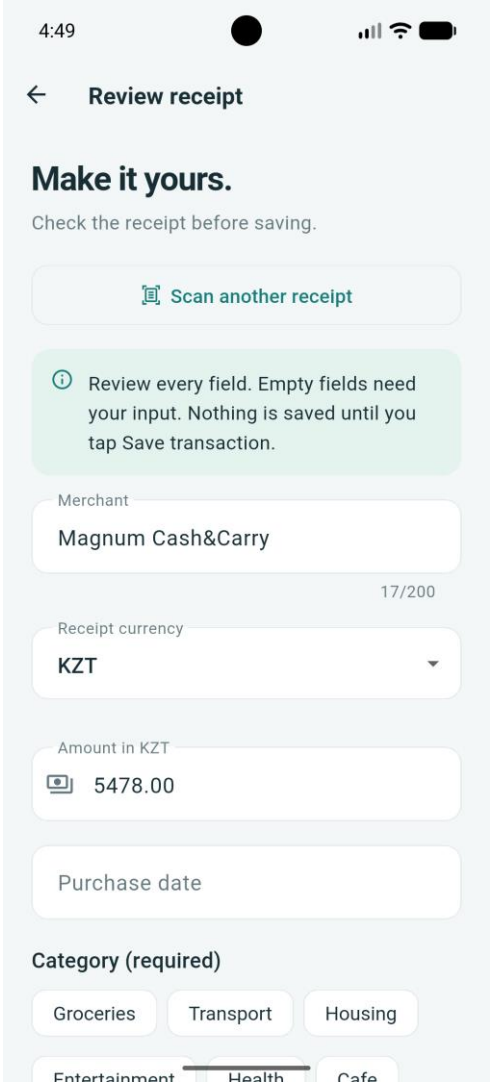


Figure 11. Review state: Magnum Cash&Carry, KZT 5478.00 and an empty purchase date.

Receipt owners confirmed consent to research use and external Gemini processing. Personal identifiers, payment-card data and raw receipt photographs are not published here. Recognition is provisional: the user must review fields and confirm before saving. The screenshot does not establish the exact provider request time or the correctness of every extracted field.

7. Validation and Saved Transactions

The supplied review screen shows validation messages for an empty purchase date and an unselected category. A later transaction-list state contains entries for Magnum Cash&Carry and “Табысты” [2].

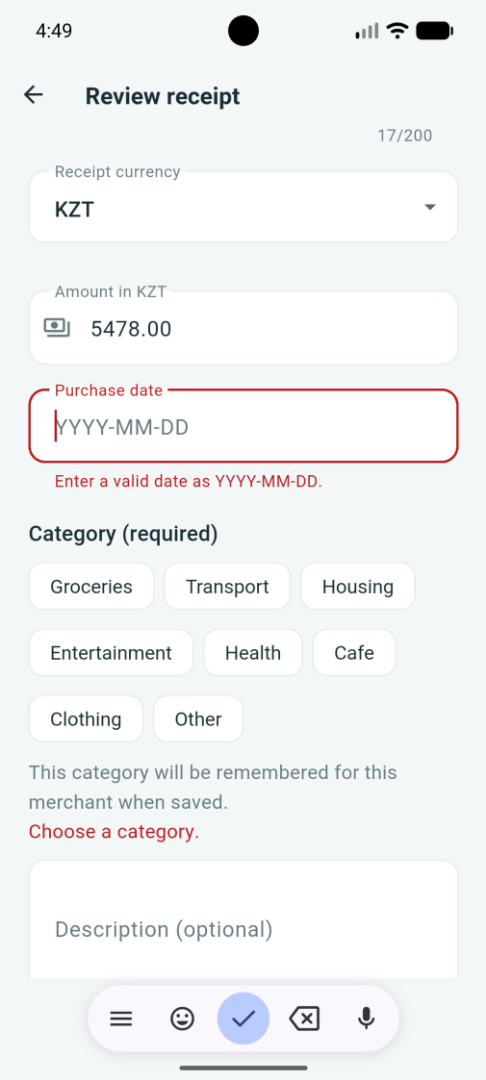


Figure 12. Required-date and category validation in the receipt review form.

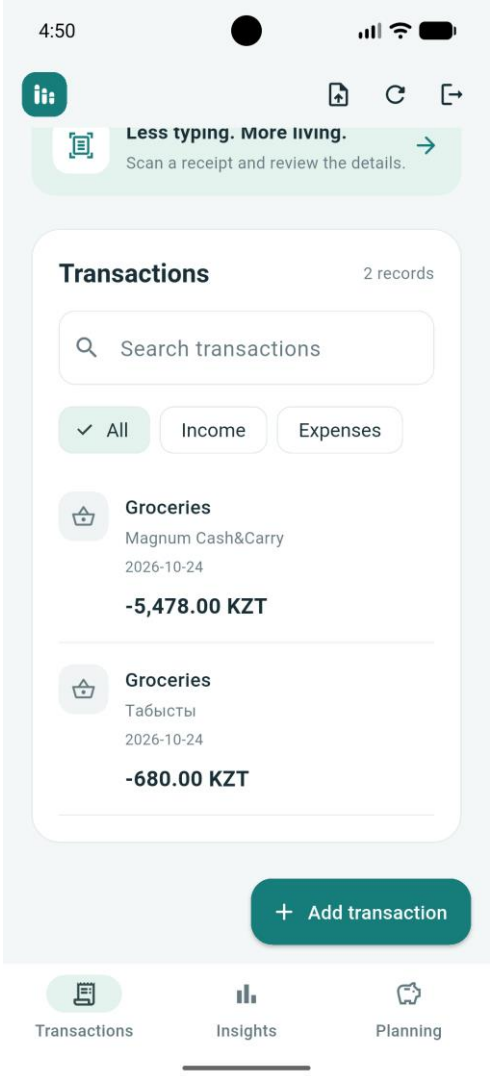


Figure 13. Later transaction list with expenses of KZT 5478.00 and KZT 680.00.

The screenshots show review and later saved-record states, but not an uninterrupted recording of the save action. The later displayed transaction date must not be treated as proof of the receipt’s original printed purchase date. Manual corrections are permitted; the research pilot uses its own reference labels.

Human review is an intentional design choice, consistent with guidance that AI-assisted systems should support correction and recovery [5]. Its effect on task completion time and user effort remains to be measured.

8. Expense Insights and Planning

The Insights screen displays recorded expense totals and category shares. The Planning screen provides budget and savings-goal sections; in the supplied state no budget has been set for the month [2].

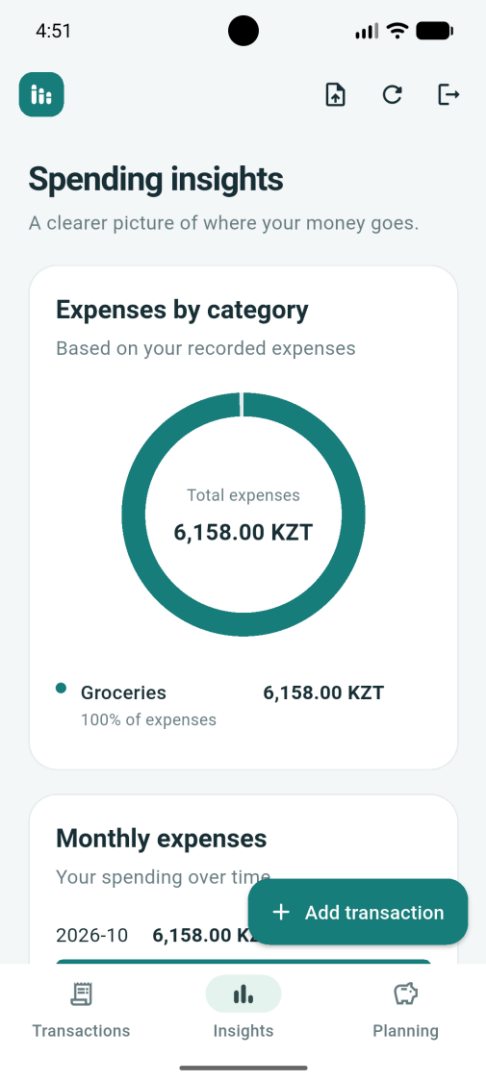


Figure 14. Insights state: total recorded expenses KZT 6158.00, all categorised as Groceries.

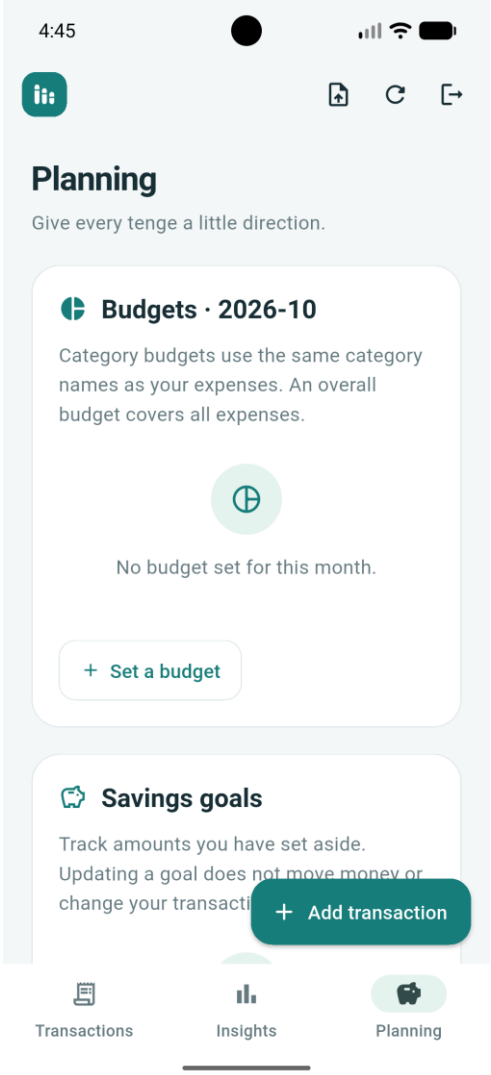


Figure 15. Planning state with no budget set and an entry point for creating one.

The Insights total is consistent with the two amounts shown in the later transaction-list screenshot. This limited consistency check does not validate every chart or establish equivalence with the older separate JavaScript client. The Planning screenshot demonstrates the empty-state interface, not a completed budget-creation or savings-goal scenario.

The source supports budgets and goals [1]. Additional end-to-end evidence is needed for budget-limit updates, overspending messages and saved-goal progress. A goal records an entered amount; it does not transfer money.

9. Testing and Preliminary Receipt Evaluation

9.1 Automated checks

The latest submitted terminal summary reports 51 tests completed in 12.972 seconds with status OK [2]. The visible test names cover analytics, authentication, user isolation, transactions, CSV handling, receipts and merchant-category memory. The full updated code and a machine-readable log for this run were not supplied; the count is reported as screenshot evidence.



Figure 16. Latest submitted unittest summary: 51 tests, 12.972 seconds, OK.

This is distinct from the earlier independent local run of the supplied archive, which passed 50 tests. The additional test cannot be identified from the final count alone. Mocked receipt tests check application behaviour, not live-provider accuracy or production readiness.

9.2 Receipt-extraction pilot

The separate submitted JSON records 16 unique receipt images, provider gemini and model gemini-3.8-flash. The two failed requests report quota exhaustion. They remain in the field-specific primary denominators [3].

Field	Matched / labelled	Match rate
Merchant	10 / 15	66.67%
Date	9 / 13	69.23%
Total	14 / 16	87.50%

Mean processing time was 29.49 s, median 29.56 s and nearest-rank 95th percentile 66.02 s. These are backend-pipeline times, not UI task times. Different field denominators reflect unknown reference labels, not different overall image sets.

The sample is small and non-random; one date label is provisional. Raw model outputs are absent, so individual mismatch causes cannot be established. The newly supplied interface screenshots are not added to this pilot’s accuracy denominators.

10. Remaining Work and Conclusion

10.1 Tasks before final defence

Priority	Task and acceptance evidence
1 · Reconcile versions	Record the commit, client build and environment used in the latest demonstration; retain a complete test log.
2 · Complete functional evidence	Demonstrate budget creation, goal updates, income entry and persistence after restart. Verify decimal-value round-trips.
3 · Evaluate analytics	Use independently labelled descriptions for accuracy and macro-F1; use chronological expense records for forecast MAE and MAPE, with an explicit zero-month rule and a simple benchmark [6].
4 · Validate the client	Run native-device checks and participant usability tasks. Compare receipt-assisted entry with manual entry, including wait and correction time.
5 · Deployment and privacy	Review storage, token handling, consent and provider failures; test PostgreSQL separately before any production-readiness claim.

10.2 Conclusion

The current materials support a working local prototype with authenticated transaction handling, relational storage, receipt review, validation and expense displays. My contribution is the client/backend integration and supporting data path, within the team allocation described above. Pre-defence evidence is strongest for the local API and demonstrated screen states. Model quality, comparative usability and deployment reliability remain separate validation tasks; no financial-benefit claim is made.

Sources and supporting materials

[1] Smart Finance project team. Supplied source archive, snapshot 8f4a9c227a67b6bbfe9a1d33df3e2c18261097d0. Repository: https://github.com/KaberDaryn/smart-finance-project.

[2] Demonstration screenshots submitted for this report: PowerShell server, health, registration, transaction requests, SQLite schema, unittest output and mobile-layout UI states. Unpublished project evidence supplied on 24 October 2026.

[3] receipt-evaluation.json. Team-supplied receipt pilot report, 16 images; used with the previously reviewed reference labels. Run timestamp not recorded.

[4] Individual project justifications for Kaber Daryn, Nuriman Aisultan and Akhmetov Dauren, supplied by the team. Role and planned-deliverable statements.

[5] Amershi, S., et al. (2019). Guidelines for Human-AI Interaction. CHI. https://doi.org/10.1145/3290605.3300233.

[6] Hyndman, R. J., and Athanasopoulos, G. (2021). Forecasting: Principles and Practice, 3rd ed., §5.2. https://otexts.com/fpp3/simple-methods.html.